
SOFTWARE REQUIREMENTS SPECIFICATION

for

AI-Enabled Telehealth and
Patient Data System

Version 1.0 approved

Process Model: Agile

Software Engineering Hackathon

May 22, 2026

Name:-	PRN No:-
Prasad Ingole	202301040064
Abhinav Shingote	202301040140
Netra Rajmane	202301040141
Disha Uplanchiwar	202301040242
Atharva Suryawanshi	202301040283
Yash Chopade	202402040032

Contents

1	Introduction	5
1.1	Purpose	5
1.2	Problem Statement	5
1.3	Project Scope	5
1.4	Intended Audience	5
1.5	Process Model: Agile	6
1.6	References	6
2	Overall Description	7
2.1	Product Perspective	7
2.2	User Classes and Characteristics	7
2.3	Operating Environment	8
2.4	Assumptions and Dependencies	8
3	External Interface Requirements	9
3.1	User Interfaces	9
3.2	Hardware Interfaces	9
3.3	Software Interfaces	9
3.4	Communications Interfaces	10
4	System Features (Functional Requirements)	11
4.1	Authentication & User Management	11
4.2	Patient Dashboard	11
4.3	Doctor Dashboard	11
4.4	Appointment Booking System	11
4.5	Video Consultation System	12
4.6	Electronic Health Records (EHR)	12
4.7	AI Symptom Checker	12
4.8	AI Emergency Detection	12
4.9	Prescription Management	13
4.10	Notification System	13
4.11	Admin Panel	13
4.12	Offline Support	13
4.13	Advanced Features (Future Scope)	13
5	Non-Functional Requirements	14
5.1	Performance Requirements	14
5.2	Security Requirements	14
5.3	Safety Requirements	14
5.4	Software Quality Attributes	15
5.5	Design Constraints	15
6	Other Requirements	16
6.1	Database Requirements	16
6.2	Internationalization Requirements	16
6.3	UML Diagrams Required	16

Appendix A: Glossary	17
Appendix B: To Be Determined List	18

Revision History

Name	Date	Reason For Changes	Version
Team	2026	Initial SRS Draft	1.0

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the requirements for the gap in rural areas by connecting patients with specialist doctors through telemedicine, AI-assisted triage, and cloud-based Electronic Health Records (EHR).

1.2 Problem Statement

In rural areas:

- Specialist doctors are unavailable and hospitals are far away
- Medical records are paper-based and difficult to access
- Emergency handling is slow, causing delayed treatment and wrong diagnosis

This system creates a digital bridge between rural patients and specialist doctors using internet connectivity, video consultation, AI support, and cloud storage.

1.3 Project Scope

The system is a web-based healthcare platform that includes:

- Patient and Doctor registration, login, and dashboards
- Appointment booking and video consultation
- AI symptom checker and emergency detection
- Electronic Health Records (EHR) management
- Prescription and report management
- Notification system and admin panel

1.4 Intended Audience

Audience	Purpose
Developers	Implementation and coding
Testers	Validation and quality assurance
Project Evaluators / Faculty	Assessment and review
System Designers	Architecture planning
Future Maintainers	Enhancements and support

1.5 Process Model: Agile

The **Agile** process model is selected because the system contains multiple independent, evolving modules. Agile supports iterative development, continuous testing, faster feedback, and flexible feature enhancement — ideal for complex healthcare platforms.

Sprint	Features
Sprint 1	Authentication — Login, Signup, JWT, Role-Based Access
Sprint 2	Patient & Doctor Dashboard — Profiles, Dashboard UI
Sprint 3	Appointment Booking — Scheduling, Slot Availability
Sprint 4	Video Consultation — WebRTC / Agora Integration
Sprint 5	EHR & Reports — Upload, Store, Download Records
Sprint 6	AI Symptom Checker — Disease Prediction, Urgency Level
Sprint 7	Notifications & Reminders — SMS, Email, Push Alerts

1.6 References

- IEEE Software Requirements Specification Standards
- React, Spring Boot, PostgreSQL, AWS Documentation
- Agora SDK Documentation
- Software Engineering Hackathon Guidelines

2 Overall Description

2.1 Product Perspective

The AI-Enabled Telehealth and Patient Data System is a self-contained web application. It is not a replacement for doctors but a digital support platform improving communication between rural patients and specialist doctors.

Layer	Components
Users	Patient — Doctor — Admin
Presentation	React Frontend, Responsive UI, Offline Form Support
API	Spring Boot REST APIs
Security & Business Logic	JWT Auth, RBAC, Appointment Mgmt, EHR, Audit Logs
Service Integration	AI Triage, Video Consultation, Notification Service
Data & Storage	PostgreSQL (AWS RDS), AWS S3 for Reports

2.2 User Classes and Characteristics

User	Main Purpose	Access Level
Patient	Enter symptoms, book consultation, view records	Own profile, appointments, prescriptions, EHR
Doctor	Consult patients, write prescriptions	Assigned patient records and appointments
Administrator	Manage system users, monitor platform	System-level management and analytics
Healthcare Assistant	Help rural patients use the system	Limited access with patient consent

2.3 Operating Environment

Component	Details
Client Devices	Smartphones, tablets, laptops, desktops
Browsers	Chrome, Firefox, Edge, Safari
Frontend	React + Vite, Tailwind CSS, Axios
Backend	Spring Boot (Java 17+)
Database	PostgreSQL on AWS RDS
Cloud Storage	AWS S3 for reports and documents
Authentication	JWT + Spring Security + BCrypt
Offline Storage	IndexedDB / Browser Local Storage
Video Consultation	Agora SDK / WebRTC
AI Service	Gemini API / OpenAI API

2.4 Assumptions and Dependencies

Assumptions

- Users have access to a smartphone, tablet, or computer with a modern browser
- Internet may be unstable in rural areas — offline support is required
- AI module provides assistance only; final decisions are made by qualified doctors
- Doctors are verified by admin before providing consultation
- Initial version focuses on core hackathon MVP features

Dependencies

- **Frontend:** React, Vite, Tailwind CSS, Axios
- **Backend:** Spring Boot, Spring Security, JPA, Hibernate
- **Database:** PostgreSQL (AWS RDS)
- **AI Service:** Gemini API or OpenAI API
- **Video:** Agora SDK / ZegoCloud / Jitsi Meet
- **Storage:** AWS S3
- **Notifications:** Email / SMS / Push APIs

3 External Interface Requirements

3.1 User Interfaces

Interface	Key Screens
Patient	Login, Registration, Symptom Entry, Appointment, Consultation, EHR, Prescriptions
Doctor	Dashboard, Appointment Queue, Patient History, Consultation, Prescription
Admin	Dashboard, User Management, Analytics, Audit Logs

UI Standards: Responsive webpage-first design, simple navigation, large buttons, clear error messages, multilingual support (English, Hindi, Marathi).

3.2 Hardware Interfaces

Side	Minimum Requirements
Patient Device	Smartphone/Tablet, 4GB RAM, front camera, mic, Android/iOS/Windows
Doctor Device	Laptop/Desktop, 8GB RAM, HD webcam, stable broadband
Server	8GB RAM, 50GB SSD, AWS EC2/RDS, HTTPS + Firewall

3.3 Software Interfaces

Component	Technology	Purpose
Frontend	React.js + Tailwind	User interface
Backend	Spring Boot	Business logic & REST APIs
Database	PostgreSQL (AWS RDS)	Structured data storage
Authentication	JWT + Spring Security	Secure auth & RBAC
Cloud Storage	AWS S3	Report and file storage
Video	Agora SDK / WebRTC	Video/audio consultation
AI Engine	Gemini / OpenAI API	Symptom analysis & triage
Offline Storage	IndexedDB / LocalStorage	Offline symptom data
Notifications	Email / SMS APIs	Reminders and alerts
OCR	Tesseract / OCR API	Report text extraction

3.4 Communications Interfaces

Protocol	Purpose
HTTPS	Secure encrypted communication
REST API (JSON)	Frontend-backend data exchange
JWT	Authentication and authorization
WebRTC	Real-time video consultation
WebSocket	Real-time notifications
SMTP	Email notification service

Consultation Fallback: Video → Audio → Chat → Store-and-Forward (Asynchronous)

4 System Features (Functional Requirements)

4.1 Authentication & User Management

Priority: High

- **FR-1:** System shall support patient, doctor, and admin signup/login.
- **FR-2:** JWT-based authentication shall be implemented.
- **FR-3:** Role-based access control shall assign different dashboards per role.
- **FR-4:** Passwords shall be hashed using BCrypt.
- **FR-5:** OTP verification shall be supported for secure login.

4.2 Patient Dashboard

Priority: High

- **FR-6:** Patient shall view and update their profile.
- **FR-7:** Patient shall upload medical reports (PDF/images).
- **FR-8:** Patient shall book, view, and cancel appointments.
- **FR-9:** Patient shall view and download digital prescriptions as PDF.
- **FR-10:** Patient shall access complete Electronic Health Records.

4.3 Doctor Dashboard

Priority: High

- **FR-11:** Doctor shall view assigned appointments and patient queue.
- **FR-12:** Doctor shall access patient medical history and EHR.
- **FR-13:** Doctor shall write and save digital prescriptions.
- **FR-14:** Doctor shall add consultation and diagnosis notes.
- **FR-15:** Doctor shall mark appointments as completed.

4.4 Appointment Booking System

Priority: High

- **FR-16:** Patient shall search and select a doctor by specialization.
- **FR-17:** System shall display available time slots.

- **FR-18:** Patient shall book, reschedule, or cancel appointments.
- **FR-19:** System shall support emergency appointment booking.
- **FR-20:** Appointment status shall be tracked in real-time.

4.5 Video Consultation System

Priority: High

- **FR-21:** System shall support secure video calling between patient and doctor.
- **FR-22:** Consultation shall fallback to audio, then chat if video fails.
- **FR-23:** Chat and file sharing shall be available during consultation.
- **FR-24:** Store-and-forward asynchronous consultation shall be supported.

4.6 Electronic Health Records (EHR)

Priority: High

- **FR-25:** System shall store patient demographics, symptoms, diagnosis, and prescriptions.
- **FR-26:** Doctors shall access EHR only for assigned patients.
- **FR-27:** EHR shall include blood group, allergies, and current medications.
- **FR-28:** Patient shall download EHR as PDF.
- **FR-29:** Audit logs shall be maintained for all EHR access events.

4.7 AI Symptom Checker

Priority: High

- **FR-30:** Patient shall enter symptoms via form, questionnaire, or voice input.
- **FR-31:** AI shall classify urgency as Low / Medium / High / Emergency.
- **FR-32:** AI shall recommend suitable doctor type or department.
- **FR-33:** Emergency symptoms (chest pain, breathing difficulty) shall trigger alerts.
- **FR-34:** Rule-based fallback shall operate when AI service is unavailable.
- **FR-35:** AI output shall clearly state it is assistance only, not a diagnosis.

4.8 AI Emergency Detection

Priority: High

- **FR-36:** System shall detect high-risk symptom combinations.
- **FR-37:** Critical cases shall be escalated with priority alerts.
- **FR-38:** Patient shall be advised to visit the nearest emergency center in critical cases.

4.9 Prescription Management

Priority: Medium

- **FR-39:** Doctor shall generate digital prescriptions with medicine details and dosage.
- **FR-40:** Patient shall download prescription as PDF.
- **FR-41:** QR-code-based prescription verification shall be supported.

4.10 Notification System

Priority: Medium

- **FR-42:** System shall send appointment reminders via SMS, email, and push notifications.
- **FR-43:** Medicine reminders shall be sent at scheduled times.
- **FR-44:** Emergency alerts shall be sent immediately upon detection.
- **FR-45:** Offline sync status shall be notified when connectivity is restored.

4.11 Admin Panel

Priority: Medium

- **FR-46:** Admin shall manage patient and doctor accounts.
- **FR-47:** Admin shall verify doctor credentials before activation.
- **FR-48:** Admin shall view analytics — consultations, emergency cases, active users.
- **FR-49:** Admin shall review audit logs for sensitive data access.

4.12 Offline Support

Priority: Medium

- **FR-50:** Patient shall enter symptoms offline using browser local storage / IndexedDB.
- **FR-51:** Data entered offline shall sync automatically when internet is restored.

4.13 Advanced Features (Future Scope)

Priority: Low

- **FR-52:** Voice-based symptom input using speech-to-text.
- **FR-53:** GPS-based nearby hospital finder and emergency navigation.
- **FR-54:** Ambulance request system with real-time tracking.
- **FR-55:** Wearable device integration (smartwatch, BP monitor).
- **FR-56:** AI health risk prediction for diabetes, heart disease, BP.
- **FR-57:** Multilingual support — English, Hindi, Marathi with voice guidance.

5 Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement	Target
NFR-1	API response time	< 2 seconds
NFR-2	Page load time on 4G webpage	< 3 seconds
NFR-3	Video consultation latency	< 150ms
NFR-4	Concurrent users supported	Min. 100 users
NFR-5	System uptime	99.5%
NFR-6	Offline sync time after reconnect	< 5 seconds

5.2 Security Requirements

- **NFR-7:** All communication shall use HTTPS encryption.
- **NFR-8:** Passwords shall be hashed with BCrypt.
- **NFR-9:** JWT tokens shall expire and require re-authentication.
- **NFR-10:** Role-based access control shall restrict data access per user type.
- **NFR-11:** Patient data shall be accessible only to the patient and authorized doctors.
- **NFR-12:** Audit logs shall record all sensitive data access events.
- **NFR-13:** Medical reports shall be encrypted in AWS S3 storage.

5.3 Safety Requirements

- **NFR-14:** AI output shall never be presented as a final medical diagnosis.
- **NFR-15:** Emergency symptoms shall trigger immediate alerts directing patient to nearest facility.
- **NFR-16:** System shall not allow unauthorized modification of medical records.
- **NFR-17:** Patient consent shall be required before sharing records with any third party.

5.4 Software Quality Attributes

Attribute	Description
Usability	Simple UI with large buttons and guided forms for rural users
Reliability	Fallback mechanisms for AI, video, and connectivity failures
Maintainability	Modular Spring Boot backend; each module independently updatable
Scalability	Cloud-based deployment supports increasing user load
Portability	Works across Android, iOS, Windows, macOS browsers
Testability	Agile sprints include unit, integration, and UAT testing each sprint

5.5 Design Constraints

- Technology stack is fixed: React frontend, Spring Boot backend, PostgreSQL database.
- System must support rural users with low-cost devices and unstable internet.
- AI module must include a rule-based fallback when external AI API is unavailable.
- Initial version is a hackathon MVP — advanced features are deferred to future versions.
- All medical decisions must be validated by a qualified doctor.

6 Other Requirements

6.1 Database Requirements

PostgreSQL on AWS RDS shall store all structured data. Key entities:

Entity	Key Attributes
Patient	patient_id, name, phone, email, blood_group, allergies
Doctor	doctor_id, name, specialization, verified, availability
Appointment	appointment_id, patient_id, doctor_id, date, status
Symptoms	symptom_id, patient_id, description, urgency_level, timestamp
Consultation	consultation_id, appointment_id, notes, type
Prescription	prescription_id, doctor_id, patient_id, medicines, dosage
EHR	ehr_id, patient_id, history, reports, updated_at
Notification	notification_id, user_id, message, type, status
AuditLogs	log_id, user_id, action, timestamp, resource_accessed

6.2 Internationalization Requirements

- System shall support English, Hindi, and Marathi languages.
- Voice guidance shall be available for users with low literacy.
- Date and time formats shall adapt to user locale.

6.3 UML Diagrams Required

The following diagrams shall be produced as part of system design:

1. Use Case Diagram — patient, doctor, admin interactions
2. Activity Diagram — appointment booking and consultation flow
3. Sequence Diagram — AI triage and video consultation
4. Class Diagram — system classes and relationships
5. ER Diagram — database entity relationships
6. Component Diagram — system modules and interfaces
7. Deployment Diagram — cloud infrastructure layout

Appendix A: Glossary

Term	Definition
EHR	Electronic Health Record — digital patient medical history
AI Triage	AI-assisted symptom analysis to classify urgency and recommend doctor
JWT	JSON Web Token — used for secure stateless authentication
RBAC	Role-Based Access Control — restricts access based on user role
WebRTC	Web Real-Time Communication — protocol for video/audio calls in browser
REST API	Representational State Transfer — standard for frontend-backend communication
IndexedDB	Browser-based database for offline data storage
AWS RDS	Amazon Relational Database Service — managed cloud PostgreSQL hosting
AWS S3	Amazon Simple Storage Service — cloud storage for files and reports
BCrypt	Password hashing algorithm for secure credential storage
MVP	Minimum Viable Product — initial version with core features only
OCR	Optical Character Recognition — extract text from uploaded images/PDFs

Appendix B: To Be Determined List

1. Final AI API selection (Gemini vs OpenAI) — TBD
2. Video SDK final selection (Agora vs ZegoCloud vs Jitsi) — TBD
3. Notification service provider (SMS gateway) — TBD
4. Deployment platform for backend (Render vs AWS EC2) — TBD
5. OCR service selection (Tesseract vs cloud OCR API) — TBD